

Competitive Programming

Section 61 Editorial: ICPC Contest Readiness

Competitive Programming Course

2026-07-10

Table of contents

1	How to read this editorial	4
2	Problem A: Scoreboard Replay	5
2.1	Restatement	5
2.2	Important constraints and traps	5
2.3	Observation	5
2.4	Sample walk-through	5
2.5	Algorithm	5
2.6	Correctness proof	6
2.7	Complexity	6
2.8	C++ implementation notes	6
2.9	Python implementation notes	6
2.10	Full C++ solution	7
2.11	Full Python solution	7
2.12	Common wrong answers	8
2.13	Tests that matter	8
3	Problem B: Time-Boxed Schedule	9
3.1	Restatement	9
3.2	Important constraints and traps	9
3.3	Observation	9
3.4	Sample walk-through	9
3.5	Algorithm	9
3.6	Correctness proof	10
3.7	Complexity	10
3.8	C++ implementation notes	10
3.9	Python implementation notes	10
3.10	Full C++ solution	10
3.11	Full Python solution	11
3.12	Common wrong answers	11
3.13	Tests that matter	12
4	Problem C: First-Round Assignment	13
4.1	Restatement	13
4.2	Important constraints and traps	13
4.3	Observation	13
4.4	Sample walk-through	13
4.5	Algorithm	13
4.6	Correctness proof	14
4.7	Complexity	14
4.8	C++ implementation notes	14
4.9	Python implementation notes	14
4.10	Full C++ solution	14
4.11	Full Python solution	15

4.12 Common wrong answers	16
4.13 Tests that matter	16
5 Stress-testing strategy	17
6 What to carry forward	18

1 How to read this editorial

These problems are deliberately simpler than a World Finals set. Their value is accuracy under contest conditions: parse the rule into state, identify a known tool despite an unfamiliar story, and prove the exact operation that makes the implementation safe.

2 Problem A: Scoreboard Replay

2.1 Restatement

Replay submissions under the problem's stated scoring rule. A team's first accepted submission for a problem increases its solved count. Its penalty for that problem is the accepted minute plus 20 for each earlier wrong submission. Later submissions to that solved problem are ignored. Finally, sort teams by more solved problems, then lower penalty, then lower id.

2.2 Important constraints and traps

The chronological order in the input matters. Wrong submissions do not add to the total penalty until that problem is accepted. A wrong submission after an acceptance has no effect. These two details are the source of most incorrect solutions.

2.3 Observation

The only information needed for one pair (`team`, `problem`) is:

```
solved[team][problem]
wrong[team][problem]
```

The pair's future behavior depends only on those two values, not on the full history. Maintain team-wide `solved_count` and `penalty` at the moment of the first acceptance.

2.4 Sample walk-through

Suppose team 2 submits W at minute 18, W at 31, and A at 47 for the same problem. The first two submissions increase `wrong[2][p]` from 0 to 2. The acceptance marks it solved, increments the solved count, and adds $47 + 20 * 2 = 87$ penalty. Any later record for (2, p) is ignored.

2.5 Algorithm

```

initialize solved, wrong, solved_count, penalty to zero
for each submission (minute, team, problem, verdict):
    if solved[team][problem]:
        continue
    if verdict is W:
        wrong[team][problem] += 1
    else:
        solved[team][problem] = true
        solved_count[team] += 1
        penalty[team] += minute + 20 * wrong[team][problem]
sort team ids by (-solved_count, penalty, id)
print each id with its final score

```

2.6 Correctness proof

We prove that the algorithm prints the required final ranking.

For a fixed team and problem, after processing any prefix of submissions, **solved** is true exactly when the prefix contains an acceptance before any later considered submission, and **wrong** equals the number of wrong submissions before that first acceptance if it has not happened yet. Initially both statements hold. A record after acceptance is correctly ignored. Before acceptance, a wrong record increases exactly that count; an accepted record adds exactly the required penalty, marks the pair solved, and prevents every later record from changing it. Thus the invariant holds by induction.

Consequently each first acceptance contributes one solved problem and exactly its stated penalty, while every other submission contributes nothing. The stored score for every team is therefore correct. The final comparator is the problem's ranking rule, so sorting by it prints the required order. $\square$

2.7 Complexity

The replay uses $O(S)$ time and $O(TP)$ memory for S submissions, T teams, and P problems. Sorting the teams costs $O(T \log T)$ time.

2.8 C++ implementation notes

Use `vector<vector<bool>>` or `vector<vector<char>>` for **solved** and `vector<vector<int>>` for **wrong** attempts. The comparator should return a tuple such as `(-solved_count[id], penalty[id], id)`; do not try to encode all ties with nested conditionals.

2.9 Python implementation notes

Nested lists make the pair state clear. `sorted(range(1, teams + 1), key=lambda team: (-solved_count[team], penalty[team], team))` implements the ranking rule directly.

2.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int teams, problems, submissions;
    if (!(cin >> teams >> problems >> submissions)) return 0;
    vector<vector<char>> solved(teams + 1, vector<char>(problems + 1));
    vector<vector<int>> wrong(teams + 1, vector<int>(problems + 1));
    vector<int> solved_count(teams + 1);
    vector<long long> penalty(teams + 1);

    while (submissions--) {
        int minute, team, problem;
        char verdict;
        cin >> minute >> team >> problem >> verdict;
        if (solved[team][problem]) continue;
        if (verdict == 'W') {
            ++wrong[team][problem];
        } else {
            solved[team][problem] = true;
            ++solved_count[team];
            penalty[team] += minute + 20LL * wrong[team][problem];
        }
    }

    vector<int> order(teams);
    iota(order.begin(), order.end(), 1);
    sort(order.begin(), order.end(), [&](int a, int b) {
        return tuple{-solved_count[a], penalty[a], a} <
            tuple{-solved_count[b], penalty[b], b};
    });
    for (int team : order) {
        cout << team << ' ' << solved_count[team] << ' ' << penalty[team] << '\n';
    }
}
```

2.11 Full Python solution

```
import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
```

```

if not data:
    return
it = iter(data)
teams = int(next(it))
problems = int(next(it))
submissions = int(next(it))
solved = [[False] * (problems + 1) for _ in range(teams + 1)]
wrong = [[0] * (problems + 1) for _ in range(teams + 1)]
solved_count = [0] * (teams + 1)
penalty = [0] * (teams + 1)

for _ in range(submissions):
    minute = int(next(it))
    team = int(next(it))
    problem = int(next(it))
    verdict = next(it)
    if solved[team][problem]:
        continue
    if verdict == b"W":
        wrong[team][problem] += 1
    else:
        solved[team][problem] = True
        solved_count[team] += 1
        penalty[team] += minute + 20 * wrong[team][problem]

order = sorted(range(1, teams + 1), key=lambda team: (-solved_count[team], penalty[team]))
print("\n".join(f"{team} {solved_count[team]} {penalty[team]}" for team in order))

if __name__ == "__main__":
    main()

```

2.12 Common wrong answers

- Add 20 penalty for every wrong submission, including unsolved problems.
- Count an acceptance twice when duplicate accepted records appear.
- Sort equal scores by input order rather than team id.

2.13 Tests that matter

Use a team with no submissions, several wrong submissions before one acceptance, a wrong submission after acceptance, tied teams, and multiple problems for one team.

3 Problem B: Time-Boxed Schedule

3.1 Restatement

Each task takes a positive number of minutes and has a deadline. Starting at minute zero, complete at most one task at a time. Find the maximum number of tasks that can all finish no later than their own deadlines.

3.2 Important constraints and traps

The target is the number of tasks, not the smallest total duration. A task is allowed to finish exactly at its deadline. The chosen tasks must be schedulable in deadline order; arbitrary input order is not meaningful.

3.3 Observation

Sort by nondecreasing deadline. When considering a task, tentatively keep it. If the total selected duration exceeds its deadline, the selected set is no longer feasible by this deadline. To preserve as many tasks as possible, discard the longest selected task: this removes one task but leaves the least possible total time among all choices of that size.

3.4 Sample walk-through

For (duration, deadline) equal to (4, 4), (3, 5), (2, 6), first keep 4. After adding 3, the total 7 exceeds deadline 5, so remove the longest duration 4 and keep only 3. Adding 2 gives total 5 by deadline 6, so two tasks remain. Choosing the first two tasks would complete only one on time; the heap makes the replacement explicit.

3.5 Algorithm

```
sort tasks by deadline
total = 0
max_heap = empty
for (duration, deadline) in sorted tasks:
    add duration to total and max_heap
    if total > deadline:
        total -= pop largest duration from max_heap
print heap size
```

3.6 Correctness proof

After processing every task whose deadline is at most d , maintain this invariant: the heap contains a feasible subset of maximum cardinality among processed tasks, and among maximum-cardinality feasible subsets it has minimum total duration.

Before the first task the invariant is immediate. Add the next task with deadline d . If the new total is at most d , the old set plus this task is feasible. No subset of processed tasks can have more than one additional task than the old maximum, so the new set has maximum cardinality; it is the only new cardinality candidate and preserves the invariant. If the total exceeds d , every feasible subset of this new size must remove at least one task. By removing a longest duration, the algorithm leaves the minimum possible total duration among all subsets formed by removing one task. If even that total were not feasible, no subset of this size could be; otherwise the remaining set is feasible and has the old maximum cardinality. The invariant holds.

At the end, the heap has maximum feasible cardinality among all tasks, which is the required answer. $\square$

3.7 Complexity

Sorting takes $O(N \log N)$. Each task is inserted and removed from the heap at most once, for $O(N \log N)$ total time and $O(N)$ memory.

3.8 C++ implementation notes

`priority_queue<long long>` is a max heap. Store the running total in `long long` because many individually small durations can sum beyond `int`.

3.9 Python implementation notes

`heapq` is a min heap. Push `-duration` and add `heapq.heappop(heap)` to the total when removing, because the popped value is negative.

3.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<pair<long long, long long>> tasks(n);
```

```

for (auto& [duration, deadline] : tasks) cin >> duration >> deadline;
sort(tasks.begin(), tasks.end(), [](auto a, auto b) { return a.second < b.second; });

priority_queue<long long> chosen;
long long total = 0;
for (auto [duration, deadline] : tasks) {
    total += duration;
    chosen.push(duration);
    if (total > deadline) {
        total -= chosen.top();
        chosen.pop();
    }
}
cout << chosen.size() << '\n';
}

```

3.11 Full Python solution

```

import heapq
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    tasks = [(data[i], data[i + 1]) for i in range(1, 2 * n + 1, 2)]
    tasks.sort(key=lambda task: task[1])
    chosen = []
    total = 0
    for duration, deadline in tasks:
        total += duration
        heapq.heappush(chosen, -duration)
        if total > deadline:
            total += heapq.heappop(chosen)
    print(len(chosen))

if __name__ == "__main__":
    main()

```

3.12 Common wrong answers

- Keep the shortest task only when a deadline is exceeded; the correct action is to remove the longest selected task.
- Sort by duration instead of deadline.

- Treat `total == deadline` as infeasible.

3.13 Tests that matter

Use one impossible task, equal deadlines, a case where replacing one long task by several short tasks is necessary, and durations whose sum needs 64 bits.

4 Problem C: First-Round Assignment

4.1 Restatement

There are teammates and unsolved problem cards. An edge means that a teammate can take ownership of that card in the first independent round. Each teammate and each card may be used at most once. Find the maximum number of cards that can receive distinct owners.

4.2 Important constraints and traps

This is not simply the number of teammates with any edge. Two teammates may both only be able to own the same card. Conversely, a teammate can be moved from an already matched card along an alternating path to make room for a new card.

4.3 Observation

Make a bipartite graph: teammates on the left and cards on the right. A valid first-round plan is exactly a matching. Use the augmenting-path DFS introduced in Section 45.

4.4 Sample walk-through

Let teammate 1 connect to cards 1 and 2, teammate 2 connect only to card 1, and teammate 3 connect to card 2 and 3. Match teammate 1 to card 1. When trying teammate 2, card 1 is occupied, but teammate 1 can move to card 2. This alternating path gives teammate 2 card 1. Teammate 3 then takes card 3, so all three receive distinct cards.

4.5 Algorithm

```
match_card[c] = no teammate for every card
answer = 0
for each teammate u:
    clear seen_card
    if augment(u):
        answer += 1

augment(u):
    for each card c adjacent to u:
        if seen_card[c]: continue
```

```

    mark seen_card[c]
    if match_card[c] is empty or augment(match_card[c]):
        match_card[c] = u
        return true
    return false

```

4.6 Correctness proof

At every point `match_card` describes a matching because a card stores at most one teammate and a teammate is reassigned only as part of a recursive alternating path. If `augment(u)` succeeds, the path starts at unmatched teammate `u` and ends at an unmatched card; reversing its matched and unmatched edges increases the matching size by one while maintaining validity.

If `augment(u)` fails, the DFS has explored every card reachable from `u` by alternating paths, so no augmenting path starting at `u` exists. By Berge's augmenting-path characterization, the current matching cannot be enlarged via that unmatched teammate. Processing all teammates therefore produces a maximum matching. Since matchings are exactly valid first-round assignments, the reported size is optimal. \square

4.7 Complexity

With L teammates and E compatibility edges, this DFS implementation takes $O(L \cdot E)$ time and $O(L + R)$ auxiliary memory, where R is the number of cards.

4.8 C++ implementation notes

Clear `seen` for each outer DFS, not once for the whole algorithm. Capturing the recursive lambda with `std::function<bool(int)>` is concise and adequate for these bounds.

4.9 Python implementation notes

The same `seen` reset is essential. Raise the recursion limit only when the input bounds justify it; the local task is deliberately small enough for a direct DFS.

4.10 Full C++ solution

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int teammates, cards, edges;
if (!(cin >> teammates >> cards >> edges)) return 0;
vector<vector<int>> graph(teammates + 1);
while (edges--) {
    int teammate, card;
    cin >> teammate >> card;
    graph[teammate].push_back(card);
}

vector<int> match_card(cards + 1);
vector<char> seen(cards + 1);
function<bool(int)> augment = [&](int teammate) {
    for (int card : graph[teammate]) {
        if (seen[card]) continue;
        seen[card] = true;
        if (match_card[card] == 0 || augment(match_card[card])) {
            match_card[card] = teammate;
            return true;
        }
    }
    return false;
};

int answer = 0;
for (int teammate = 1; teammate <= teammates; ++teammate) {
    fill(seen.begin(), seen.end(), false);
    answer += augment(teammate);
}
cout << answer << '\n';
}

```

4.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    teammates, cards, edges = data[:3]
    graph = [[] for _ in range(teammates + 1)]
    index = 3
    for _ in range(edges):
        teammate, card = data[index], data[index + 1]
        index += 2
        graph[teammate].append(card)

    match_card = [0] * (cards + 1)

```

```

def augment(teammate: int) -> bool:
    for card in graph[teammate]:
        if seen[card]:
            continue
        seen[card] = True
        if match_card[card] == 0 or augment(match_card[card]):
            match_card[card] = teammate
            return True
    return False

answer = 0
for teammate in range(1, teammates + 1):
    seen = [False] * (cards + 1)
    answer += augment(teammate)
print(answer)

if __name__ == "__main__":
    main()

```

4.12 Common wrong answers

- Assign each teammate greedily to its first available card.
- Forget that a currently matched teammate may move to another card.
- Keep `seen` from the previous teammate's search.

4.13 Tests that matter

Use no edges, more teammates than cards, duplicate edges, and an augmenting path that needs to rematch at least two teammates.

5 Stress-testing strategy

For A, compare the state simulation to a slow per-team/per-problem replay on small random histories. For B, enumerate every subset for $N \leq 18$, sort each subset by deadline, and compare its feasible count. For C, enumerate all assignments on very small bipartite graphs or compare with a max-flow model. The oracle must be independently simpler than the optimized implementation.

6 What to carry forward

The final competitive advantage is not a magical template. It is a team that can make its model, invariant, blocker, and next test visible. Repeat the simulation protocol in `PRACTICE.md`, then use its postmortem to choose the next individual training block.